

BLUE INFORMATION TECHNOLOGY

University Field Training Program

Full-Stack Development

TASK 08 - REQUIREMENTS

Vue Router, Dynamic Routes, Reusable Composables & SPA Navigation

Day 8 | 8 Practical Training Hours

August 11, 2026

Submission: By the end of the training day

Task Number	08
Training Track	Vue Router - SPA Navigation, Dynamic Routes, Composables, and Route-Aware API State
Task Duration	8 practical training hours (approximately 6 hours hands-on implementation)
Task Date	August 11, 2026
Submission Deadline	By the end of the training day
Submission Method	Reply to the task email with repository/live links, required screenshots, latest commit, and progress update

1. Task Overview

Task 08 continues the Vue.js application created in Task 07. You will keep working inside the same task-07-vue-app folder and convert the current single-page layout into a routed Single Page Application (SPA). The task introduces Vue Router, route-based page structure, dynamic route parameters, reusable composables for API logic, and route-aware search behavior while continuing to use the JSONPlaceholder posts API.

Important - Continue the Same Vue Application

Do not create a new Vue project for Task 08. Continue inside task-07-vue-app and preserve the components and API work completed in Task 07. Today you are reorganizing that application into route-level views and extracting reusable logic, not restarting the project.

2. Objective

By the end of this task, you should be able to configure Vue Router in a Vue 3 + Vite application, organize the application into route-level views, navigate without full page reloads, create dynamic detail routes, read and use route parameters, move reusable API logic into a composable, and test navigation behavior across refresh, browser history, desktop, tablet, and mobile.

3. Expected Learning Outcomes

- Configure Vue Router in the existing Vue 3 application and understand the role of the router instance, RouterLink, and RouterView.
- Organize a Vue application into reusable layout components and route-level views instead of rendering the complete application from App.vue.
- Define static and dynamic routes, read route parameters with `useRoute()`, and perform programmatic navigation with `useRouter()`.
- Extract reusable API state and request logic into a Vue composable and reuse it from route-level components.
- Test route navigation, browser back/forward behavior, direct route access, 404 handling, responsive layout, and deployed SPA behavior.

4. Part 1 - Vue Router Setup and Application Structure

Continue from the Vue application completed in Task 07. Do not create another Vue project and do not delete the components, API work, or styling already implemented. The goal is to reorganize the existing application around routes while preserving the working functionality from yesterday.

- Install Vue Router in task-07-vue-app and add it to the existing project dependencies.
- Create `src/router/index.js` and define the router configuration in that file instead of placing route definitions directly in `main.js`.
- Register the router with the Vue application before mounting the app, and confirm that navigation works without full page reloads.
- Create a `src/views/` folder for route-level pages and keep reusable UI pieces inside `src/components/`.
- Use Vue 3 Composition API with `<script setup>`. Keep `App.vue` focused on the shared application shell and `RouterView` rather than page-specific business logic.
- Commit the router setup and initial view structure separately before implementing dynamic routes and API refactoring.

Suggested structure:

```
task-07-vue-app/
├── src/
│   ├── assets/
│   ├── components/
│   │   ├── SiteHeader.vue
│   │   ├── ServiceCard.vue
│   │   └── PostCard.vue
│   ├── composables/
│   │   └── usePosts.js
│   ├── router/
│   │   └── index.js
│   └── views/
│       ├── HomeView.vue
│       ├── ServicesView.vue
│       ├── PostsView.vue
│       ├── PostDetailsView.vue
│       ├── ContactView.vue
│       └── NotFoundView.vue
├── App.vue
├── main.js
└── package.json
```

5. Part 2 - SPA Pages, Navigation, and Route-Level Views

Convert the current Vue page into a small Single Page Application. Reuse the components and design direction from Task 07, but organize the application so that different sections are accessible through dedicated routes.

- Create at least four route-level pages: Home, Services/Projects, Posts, and Contact. You may add an About page if it fits your existing design.
- Use RouterLink for internal application navigation. Internal links should not trigger a complete browser reload.
- Render route-level content through RouterView and keep the site header/navigation visible across routes.
- Add a clear active navigation state so users can identify the current route. The navigation must continue to work on desktop, tablet, and mobile.
- Lazy-load at least two route-level view components using dynamic imports so route components are loaded only when needed.

6. Part 3 - Dynamic Routes, Route Parameters, and Post Details

- Add a dynamic post details route such as `/posts/:id` and connect it to a dedicated `PostDetailsView.vue` component.
- Update the posts list so every post has a "View Details" or "Read More" action that navigates to the correct dynamic route using the post id.
- In `PostDetailsView.vue`, read the route parameter using `useRoute()` and use the id to determine which post must be loaded.
- Load the selected post from `https://jsonplaceholder.typicode.com/posts/{id}` and display the title, body, and id in a clear detail's layout.
- Provide loading, API error, and invalid/not-found handling. Add a Back to Posts action and use router navigation rather than relying only on the browser back button.
- Add a catch-all Not Found route for unknown application URLs. Test a valid post route, an invalid post id, and an unknown route before submission.

Required navigation flow:

```

PostsView
-> renders post cards
RouterLink /posts/:id
-> navigates without full reload
PostDetailsView
-> reads route.params.id
Composable / API function
-> loads the selected post
Template
-> renders loading, error, or post details
  
```

7. Part 4 - Reusable Composable and Route-Aware API State

Refactor the API logic created in Task 07 so request state and reusable request behavior are not duplicated across route-level components. Create a composable inside `src/composables/` and use it from the posts-related views.

- Create a composable such as `usePosts.js`, `usePost.js`, or a small generic `useFetch.js`. The filename and structure may vary, but the reusable logic must be outside the view component.
- The composable should expose the reactive data needed by the UI, including data, loading state, error state, and a reusable load/retry function.
- Use the composable from `PostsView` and/or `PostDetailsView` instead of copying the complete fetch/loading/error logic into both views.
- Keep the `JSONPlaceholder` base URL in one clear location or reusable constant rather than repeating the complete URL unnecessarily throughout the application.
- Preserve the title search/filter behavior from Task 07. Store the current search value in the URL query string, for example `/posts?q=vue`, and restore the filter when the route is opened with that query.
- Update the query string without performing a full page reload. Verify that refreshing or sharing the route keeps the intended search value available.

8. Part 5 - Routing Rules, Testing, and Code Quality

- Use `Vue Router` for application navigation. Do not manually show/hide complete pages using DOM selectors or a large set of boolean variables.
- Do not introduce `Pinia`, `Vuex`, `Nuxt`, `Axios`, `jQuery`, or another state-management/framework layer in this task. Shared state management will be introduced later when required.
- Test `RouterLink` navigation, browser Back/Forward, direct access to a valid route, dynamic route parameters, the Not Found page, and query-string restoration.
- Test the application after refresh and at desktop, tablet, and mobile widths. Resolve console errors, Vue warnings, failed imports, and broken links before submission.
- If your static hosting environment does not support SPA history fallback, document the behavior and use an appropriate router history strategy for the deployed build so route refreshes remain usable.
- Update `README.md` with a "Task 08 - Vue Router & SPA Navigation" section covering routes, dynamic-route behavior, composable structure, API usage, deployment/history decision, and any known limitation.

9. Estimated Work Plan

Estimated Time	Required Work
45 min	Install/configure Vue Router, create router/views structure, and register the router in the existing Vue app.
75 min	Convert the current application into route-level pages with RouterLink, RouterView, shared navigation, active states, and lazy-loaded views.
90 min	Implement /posts/:id, route parameters, post details loading, back navigation, invalid-id handling, and the Not Found route.
90 min	Extract reusable API logic into a composable and synchronize the posts search value with the route query string.
30 min	Test navigation, Back/Forward, refresh/direct routes, responsive behavior, error states, and console output.
30 min	Update README, capture screenshots, commit/push the final version, and prepare the end-of-day report.

Implementation Scope

The required hands-on implementation is planned for approximately 6 hours. Use the remaining training time for reviewing Vue Router and Composition API documentation, debugging route behavior, testing deployed refresh behavior, improving code quality, and documenting what you learned.

10. Git and Commit Requirements

- Continue using the same public blue-fullstack-training repository.
- Continue working inside task-07-vue-app. Do not create a separate task-08 Vue project unless explicitly instructed.
- Create multiple meaningful commits during the day, including separate commits for router setup, dynamic routes, and API/composable refactoring.
- Push the latest tested version to GitHub before sending the end-of-day report.

Examples of acceptable commit messages:

```
feat: configure Vue Router and route-level views
feat: add SPA navigation and active route states
feat: add dynamic post details route
refactor: extract posts API logic into composable
feat: sync post search with route query
docs: update README for task 08
```

11. Required Deliverables

- Updated public GitHub repository with Task 08 implemented inside the existing task-07-vue-app application.
- Working Vue Router setup with at least four route-level pages, shared navigation, active-route state, and a Not Found route.
- Working `/posts/:id` dynamic route with post details, loading/error/not-found handling, and navigation back to the posts list.
- Reusable composable-based API logic and route-aware post search using a query string.
- Updated README documentation for Task 08, including the routing structure and deployed history/fallback decision.

12. Acceptance Criteria

Criterion	Expected Result
Router setup	Vue Router is installed and registered correctly in the existing task-07-vue-app application.
Route structure	At least four route-level pages render through RouterView and shared navigation uses RouterLink with an active state.
Dynamic route	<code>/posts/:id</code> loads the requested post using the route id and includes clear navigation back to the posts list.
Error / 404 handling	Invalid post requests and unknown routes show a controlled state instead of a blank or broken page.
Composable	Reusable API loading/error/data logic is extracted outside the route view and used by the posts-related UI.
Route-aware search	The posts search value is represented in the route query and can be restored from the URL.
Quality	Navigation, Back/Forward, refresh/deployment behavior, responsive layouts, README, console output, and Git history are verified.

13. End-of-Day Submission

Reply to the Task 08 email by the end of the training day and include:

- The public GitHub repository link.
- A screenshot showing navigation between at least two route-level pages and the active navigation state.
- A screenshot showing a working dynamic post details route with the post id visible in the URL.
- A screenshot showing either the route-aware posts search/query string or the Not Found/error state.
- A short summary of completed work and any remaining work.
- Any challenges, blockers, or questions you faced.
- Your latest commit message or commit reference.

Progress Item	What to Include
Completed	Summarize the router setup, route-level pages, dynamic route, composables, and query-string work completed today.
Remaining	List any unfinished requirement, or write None.
Challenges / Blockers	Describe any issue that affected route/API progress, or write None.
Questions	Add anything that needs clarification, or write None.
GitHub Link	Paste the public blue-fullstack-training repository URL.
Evidence / Latest Commit	Attach the required screenshots and add the latest commit message/reference.

14. Weekly Feedback Process

Weekly Consolidated Feedback

Task 08 is part of Week 02. Continue submitting the daily progress update after each task. Consolidated feedback will be provided at the end of the week and will cover all tasks submitted during the week. Apply the requested corrections after receiving the weekly feedback.

15. Final Checklist Before Submission

- *I continued inside task-07-vue-app and preserved the working functionality from Task 07.*
- *Vue Router is installed and configured, and route-level pages render through RouterView.*
- *I created at least four routes plus a Not Found route, and internal navigation uses RouterLink.*
- *The /posts/:id dynamic route loads and displays the correct post using the route parameter.*
- *API request state is extracted into a reusable composable rather than duplicated across views.*
- *Posts search is reflected in the route query string and restores correctly when the route is opened again.*
- *Navigation, Back/Forward, refresh behavior, responsive layouts, error states, and console output have been tested.*
- *README is updated and the latest tested code is committed and pushed to GitHub.*